DATA 255 Deep Learning and Computer Vision in Applied Data Intelligence

Feb 18, 2026

ISB 870

Taehee Jeong, Ph.D.

SJSU SAN JOSÉ STATE UNIVERSITY

Previous Topics

- Deep neural networks

Forward propagation

Input:
- data

→

Neural Network:
- Weights(parameters)
- Architecture
- Functions(activation, etc.)

→

Output:
- Predictions

Neural Network: 1 hidden layer

$$a_1^{[0]} \rightarrow a^{[1]} \rightarrow a^{[2]} \rightarrow \hat{y}$$

Input layer Hidden layer Output layer

$$a_1^{[0]}, W_1^{[1]}, W_0^{[1]} \rightarrow \boxed{Z^{[1]} = W_0^{[1]} + W_1^{[1]} \cdot a_1^{[0]}} \rightarrow \boxed{a^{[1]} = \sigma(Z^{[1]})} \rightarrow \boxed{Z^{[2]} = W^{[2]} a^{[1]} + W_0^{[2]}} \rightarrow \boxed{a^{[2]} = \sigma(Z^{[2]})} \rightarrow \hat{y}$$

$$\hat{y} = \begin{cases} 1 \; if \, a^{[2]} \geq 0.5 \\ 0 \; if \, a^{[2]} < 0.5 \end{cases}$$

Forward propagation

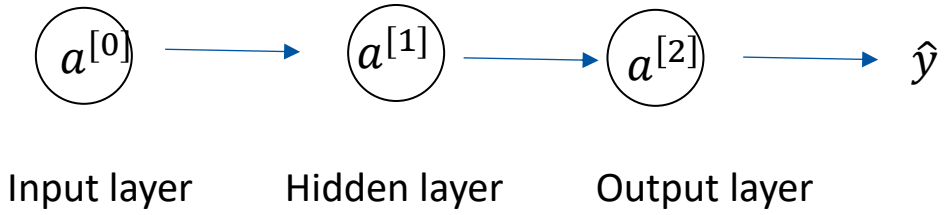
$$Z^{[1]} = W^{[1]} \cdot a^{[0]} + W_0^{[1]}$$

$$a^{[1]} = \sigma(Z^{[1]})$$

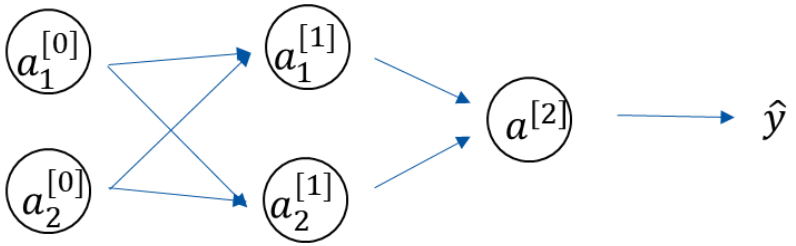
$$Z^{[2]} = W^{[2]} \cdot a^{[1]} + W_0^{[2]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$\hat{y} = \begin{cases} 1 \; if \; a^{[2]} \geq 0.5 \\ 0 \; if \; a^{[2]} < 0.5 \end{cases}$$



Forward propagation



Input layer Hidden layer Output layer

1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} \cdot a_1^{[1]} + W_2^{[2]} \cdot a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$\hat{y} = \begin{cases} 1 \; if \; a^{[2]} \geq 0.5 \\ 0 \; if \; a^{[2]} < 0.5 \end{cases}$$

Today's Topics

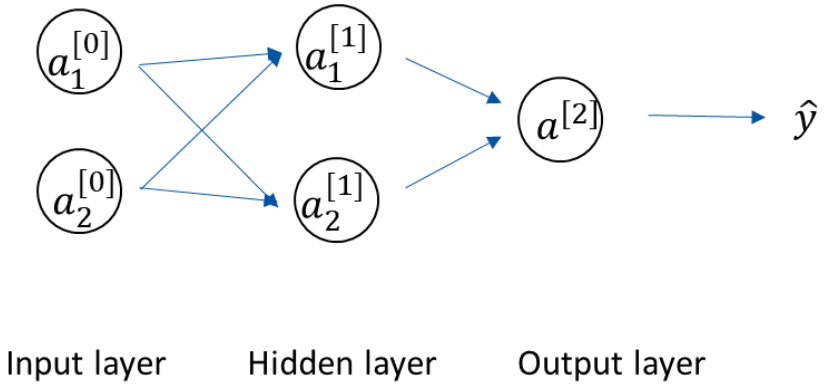
- Model Optimization Procedure
- Gradient Descent
- Backward Pass for binary classification

Model optimizing procedure

1. Initialize parameters
2. Run the optimization loop
    1. Forward propagation to compute the loss function
    2. Backward propagation to compute the gradients with respect to the loss function
    3. Using the gradients, update your parameter with the gradient descent update rule
3. Return the learned parameters

Source: deep learning, Andrew Ng

Forward propagation



$$a_1^{[0]}, a_2^{[0]}$$

$$\downarrow$$

$$Z_1^{[1]}, Z_2^{[1]}$$

$$\downarrow$$

$$a_1^{[1]}, a_2^{[1]}$$

$$\downarrow$$

$$Z^{[2]}$$

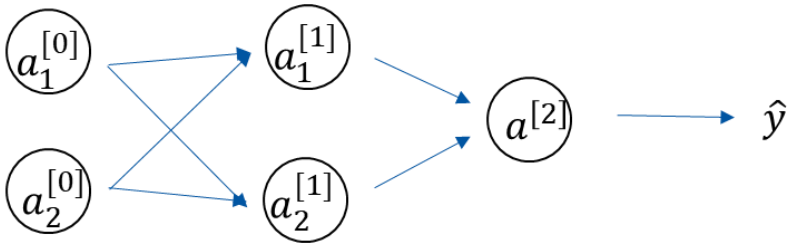
$$\downarrow$$

$$a^{[2]}$$

$$\downarrow$$

$$\hat{y}$$

Forward propagation



Input layer Hidden layer Output layer

1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$\hat{y} = \begin{cases} 1 \; if \; a^{[2]} \geq 0.5 \\ 0 \; if \; a^{[2]} < 0.5 \end{cases}$$

Cost function: binary cross entropy

$$J = -\frac{1}{m}\sum_{i=1}^{m}[y^i \log\left(a^{[2]}\right) + \left(1 - y^i\right)\log(1 - a^{[2]})]$$

Gradient Descent

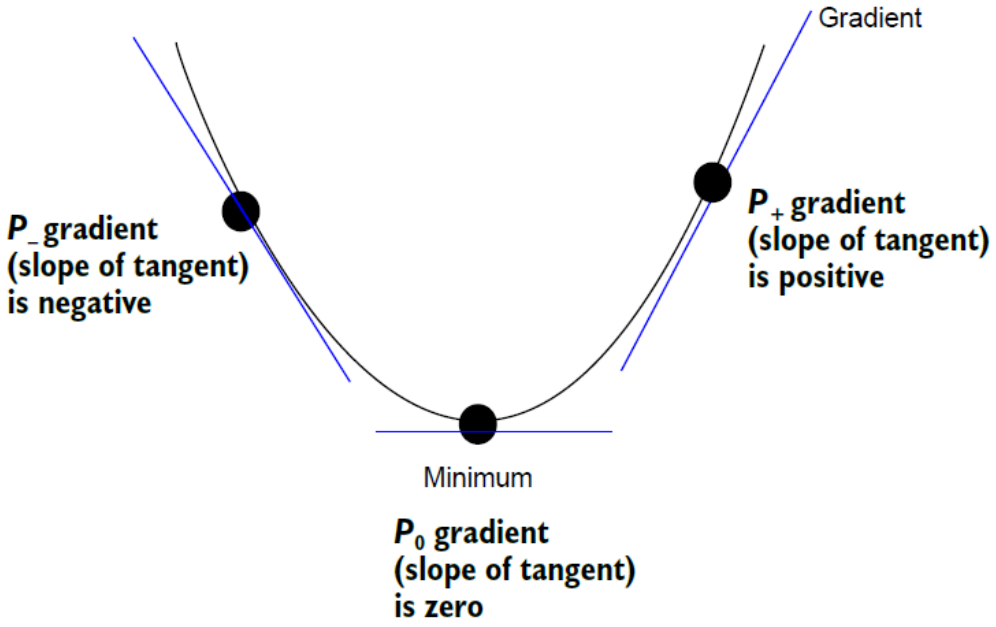
Find optimum W to minimize J

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

$$J = \frac{1}{m} \sum_{i=1}^{m} L(\hat{y}, y)$$

- *W : weight or Model parameters*
- *J : cost function*
- *L: loss function(cross entropy)*
- α: *learning rate or step size*
- $\frac{\partial J}{\partial W}$: gradient

Gradient is zero at the minimum

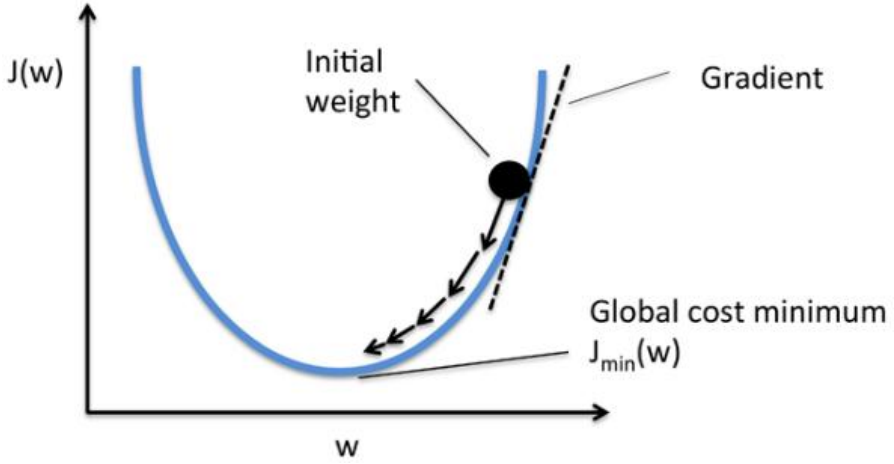


Any optimum (that is, maximum or minimum) of a function is a point of inflection.
This means the function turns around at the optimum point.

Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Gradient descent method

- To find the weights that minimize a cost function, gradient descent is commonly used as an optimization algorithm.
- Gradient(vector of partial derivatives) gives the input direction in which the function **decreases**
  - Step0: pick a random starting point
  - Step1: compute the gradient
  - Step2: take a small step in the direction of the gradient
  - Step3: repeat with the new starting point



Source: https://www.bogotobogo.com/python/scikit-learn/scikit-learn_batch-gradient-descent-versus-stochastic-gradient-descent

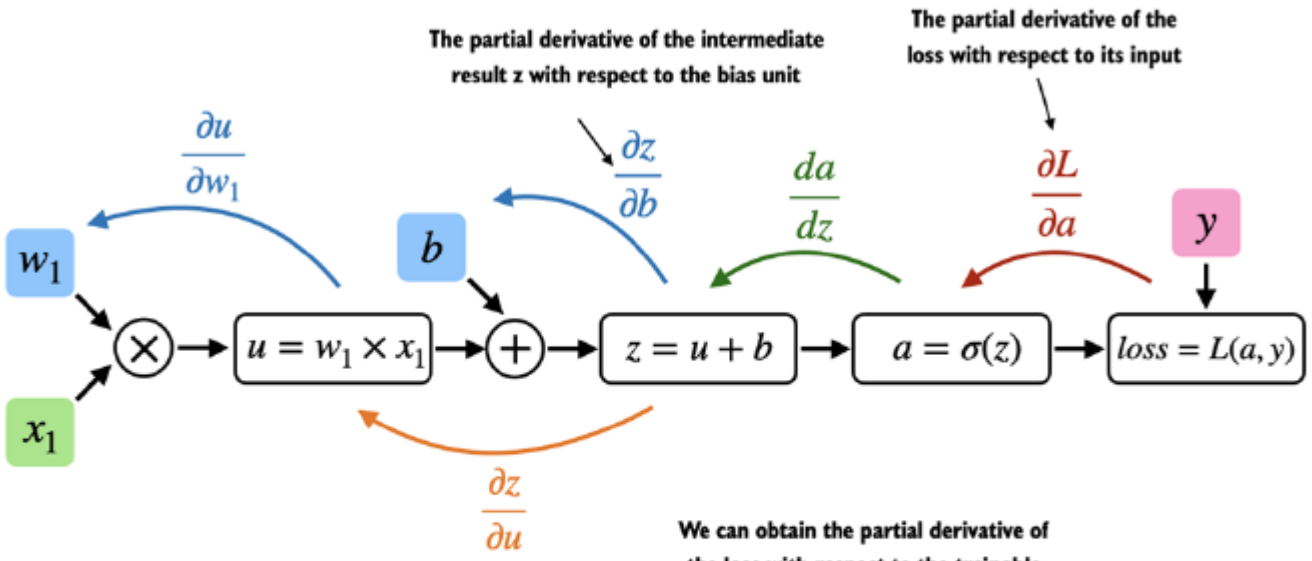
Backward propagation

- The derivative of the loss in forward pass needs to be computed with respect to the weights in all layers in the backwards pass.
- The main goal of the backward propagation is to learn the gradient of the loss function with respect to the different weights by using the chain rule.
- These gradients are used to update the weights.
- Since these gradients are learned in the backward direction, starting from the output node, this learning process is referred to as the backward pass.

Source: Neural Networks and Deep Learning, Charu C. Aggarwal

Backpropagation

Backpropagation is a way of computing gradients of expressions through recursive application of chain rule.



$$\frac{\partial L}{\partial w_1} = \frac{\partial u}{\partial w_1} \times \frac{\partial z}{\partial u} \times \frac{da}{dz} \times \frac{\partial L}{\partial a}$$

We can obtain the partial derivative of the loss with respect to the trainable weight by chaining the individual partial derivative in the graph

$$\frac{\partial L}{\partial b} = \frac{\partial z}{\partial b} \times \frac{da}{dz} \times \frac{\partial L}{\partial a}$$

Similar to above, we can compute the partial derivative of the trainable derivative by applying the chain rule

Source: Build a Large Language Model (from Scratch), Raschka, Sebastian

Derivative formula

Product rule

$h(x) = f(x) \cdot g(x)$

$$\frac{d}{dx}\, h(x) = \left[\frac{d}{dx} f(x)\right] \cdot g(x) + f(x) \cdot [\frac{d}{dx} g(x)]$$

$h(x) = f(g(x))$

$$\frac{d}{dx}[f(g(x))] = f'\big(g(x)\big) \cdot g'\,(x)$$

Chain rule

If $\mathrm{y = f(u)}$, where $\mathrm{u = g(x)}$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Derivative formula

Product rule

$$h(x) = f(x) \cdot g(x)$$

$$\frac{\partial h}{\partial x} = f' \cdot g + f \cdot g'$$

$$h(x) = \frac{f(x)}{g(x)}$$

$$h = f \cdot g^{-1}$$

$$\frac{\partial h}{\partial x} = f' \cdot g^{-1} + f \cdot \{-g^{-2}\} \cdot g'$$

$$= \frac{f'}{g} - \frac{f \cdot g'}{g^2}$$

$$= \frac{f' \cdot g - f \cdot g'}{g^2}$$

Derivatives of Sigmoid function

$$g(z) = \frac{1}{1 + \exp(-z)}$$

$$\frac{d}{dx} g(x) = \frac{\exp(-x)}{(1 + \exp(-x))^2}$$

$$g'(z) = g(z)(1 - g(z))$$

$$\frac{d}{dx} e^x = e^x$$

$$\frac{d}{dx} e^{-x} = -e^{-x}$$

Derivatives of Tanh function

$$g(z) = tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$

$$g'(z) = 1 - g(z)^{\,2}$$

$$\frac{d}{dx} e^{x} = e^{x}$$

$$\frac{d}{dx} e^{-x} = -e^{-x}$$

** You should not use derivatives of trigonometric functions.

Derivatives of ReLU and Leaky ReLU

ReLU

$$a = g(z) = \max(0, z)$$

$$g'(z) = \begin{Bmatrix} 0 & if\ z < 0 \\ 1 & if\ z \geq 0 \end{Bmatrix}$$

Leaky ReLU

$$a = g(z) = \max(0.01z, z)$$

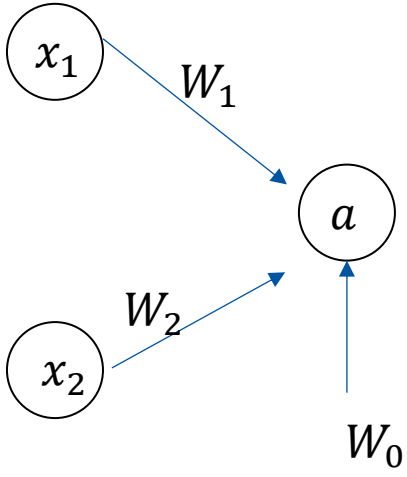
$$g'(z) = \begin{Bmatrix} 0.01 & if\ z < 0 \\ 1 & if\ z \geq 0 \end{Bmatrix}$$

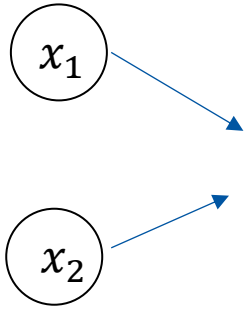
Derivatives of ELU

$$a = ELU(z) = \begin{Bmatrix} \alpha(\exp(z) - 1) & if\ z < 0 \\ z & if\ z \geq 0 \end{Bmatrix}$$

$$g'(z) = \begin{Bmatrix} \alpha \exp(z) & if\ z < 0 \\ 1 & if\ z \geq 0 \end{Bmatrix}$$

Logistic regression





$$\boxed{Z = W_0 + W_1 x_1 + W_2 x_2} \longrightarrow \boxed{a = \sigma(Z)}$$

$$\downarrow$$

$$\hat{y}$$

$$\hat{y} = \begin{cases} 1 \; if \; a^{[2]} \geq 0.5 \\ 0 \; if \; a^{[2]} < 0.5 \end{cases}$$

$$L(\hat{y}, y) = -[y * \log(a) + (1 - y) * \log(1 - a)]$$

Derivative of binary cross entropy

$$L(a, y) = -[y\log a + (1-y)\log(1-a)]$$

$$\frac{\partial L}{\partial W_0} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial Z}\frac{\partial Z}{\partial W_0}$$

$$\frac{\partial L}{\partial W_0} = \left[-\frac{y}{a} + \frac{1-y}{(1-a)}\right][a * (1-a)] \cdot 1$$

$$\frac{\partial L}{\partial W_0} = [-y(1-a) + a(1-y)] \cdot 1$$

$$\frac{\partial L}{\partial W_0} = (a-y) \cdot 1$$

$$\frac{d}{dx}\, log(x) = \frac{1}{\mathrm{x}}$$

$$\frac{d}{dx}\, log(1-x) = \frac{-1}{1-\mathrm{x}}$$

$$a = \sigma(z)$$

$$\frac{d}{dz}\sigma(z) = \sigma(z) * [1-\sigma(z)]$$

$$Z = W_0 + W_1 x_1 + W_2 x_2$$

Derivative of binary cross entropy

$$L(a, y) = -[y\log a + (1-y)\log(1-a)]$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial Z}\frac{\partial Z}{\partial W_1}$$

$$\frac{\partial L}{\partial W_1} = \left[-\frac{y}{a} + \frac{1-y}{(1-\hat{y})}\right][a * (1-a)] \cdot x_1$$

$$\frac{\partial J}{\partial W_1} = [-y(1-a) + a(1-y)] \cdot x_1$$

$$\frac{\partial J}{\partial W_1} = (a-y) \cdot x_1$$

$$\frac{d}{dx}\, log(x) = \frac{1}{\mathrm{x}}$$

$$\frac{d}{dx}\, log(1-x) = \frac{-1}{1-\mathrm{x}}$$

$$a = \sigma(z)$$

$$\frac{d}{dz}\sigma(z) = \sigma(z) * [1-\sigma(z)]$$

$$Z = W_0 + W_1 x_1 + W_2 x_2$$

Derivative of binary cross entropy

$$L(a, y) = -[y\log a + (1-y)\log(1-a)]$$

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial Z}\frac{\partial Z}{\partial W_2}$$

$$\frac{\partial L}{\partial W_2} = \left[-\frac{y}{a} + \frac{1-y}{(1-a)}\right][a * (1-\hat{y})] \cdot x_2$$

$$\frac{\partial J}{\partial W_2} = [-y(1-a) + a(1-y)] \cdot x_2$$

$$\frac{\partial J}{\partial W_2} = (a-y) \cdot x_2$$

$$\frac{d}{dx}\, log(x) = \frac{1}{\mathrm{x}}$$

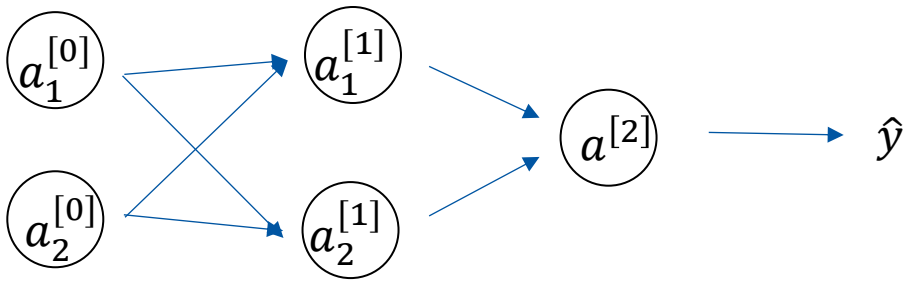
$$\frac{d}{dx}\, log(1-x) = \frac{-1}{1-\mathrm{x}}$$

$$a = \sigma(z)$$

$$\frac{d}{dz}\sigma(z) = \sigma(z) * [1-\sigma(z)]$$

$$Z = W_0 + W_1 x_1 + W_2 x_2$$

Neural Network: 1 output neuron



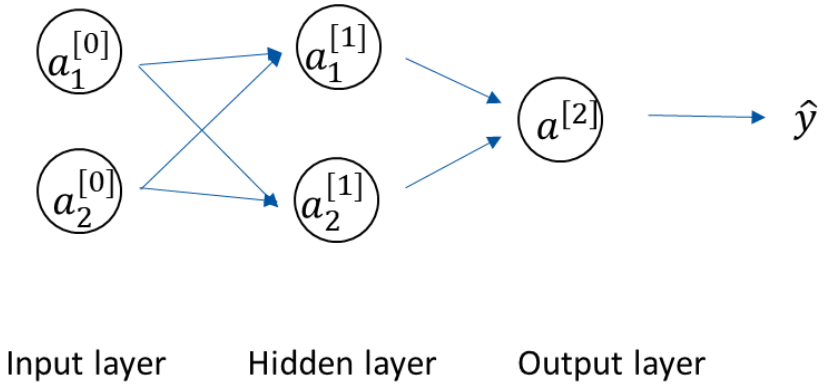
Input layer Hidden layer Output layer

Parameters: $W^{[1]}$, $W^{[2]}$

Cost function: $L(W^{[1]}, W^{[2]})$

$$J = \frac{1}{m}\sum_{i=1}^{m} L(a^{[2]}, y)$$

Forward propagation



$a_1^{[0]}, a_2^{[0]}$

$\downarrow$

$Z_1^{[1]}, Z_2^{[1]}$

$\downarrow$

$a_1^{[1]}, a_2^{[1]}$

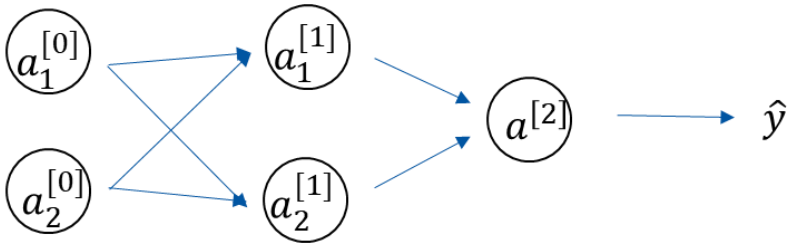
$\downarrow$

$Z^{[2]}$

$\downarrow$

$a^{[2]}$

Forward propagation



Input layer Hidden layer Output layer

1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$\hat{y} = \begin{cases} 1 \; if \; a^{[2]} \geq 0.5 \\ 0 \; if \; a^{[2]} < 0.5 \end{cases}$$

Derivative of $Z^{[2]}$

backpropagation for binary entropy loss

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

$$L\left(a^{[2]}, y\right) = -[y * \log\left(a^{[2]}\right) + (1 - y) * \log\left(1 - a^{[2]}\right)]$$

$$dZ^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}}$$

$$dZ^{[2]} = (a^{[2]} - y)$$

**Element wise multiplication*

Derivative of $W_0^{[2]}$

backpropagation for binary entropy loss

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

$$L\left(a^{[2]}, y\right) = -[y * \log\left(a^{[2]}\right) + (1-y) * \log\left(1 - a^{[2]}\right)]$$

$$\frac{\partial L(a,y)}{\partial W_0^{[2]}} = dW_0^{[2]}$$

$$dW_0^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_0^{[2]}}$$

$$= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_0^{[2]}}$$

$$dW_0^{[2]} = dZ^{[2]} \cdot 1$$

**Element wise multiplication*

Derivative of $W_1^{[2]}$

backpropagation for binary classification

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

$$L\left(a^{[2]}, y\right) = -[y * \log\left(a^{[2]}\right) + (1-y) * \log\left(1 - a^{[2]}\right)]$$

$$\frac{\partial L(a^{[2]}, y)}{\partial W_1^{[2]}} = dW_1^{[2]}$$

$$dW_1^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_1^{[2]}}$$

$$= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_1^{[2]}}$$

$$dW_1^{[2]} = dZ^{[2]} \cdot a_1^{[1]}$$

**Element wise multiplication*

SJSU SAN JOSÉ STATE UNIVERSITY

Derivative of $W_2^{[2]}$

backpropagation for binary classification

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

$$L\left(a^{[2]}, y\right) = -[y * \log\left(a^{[2]}\right) + (1 - y) * \log\left(1 - a^{[2]}\right)]$$

$$\frac{\partial L(a^{[2]}, y)}{\partial W_2^{[2]}} = dW_2^{[2]}$$

$$dW_2^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_2^{[2]}}$$

$$= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_2^{[2]}}$$

$$dW_2^{[2]} = dZ^{[2]} \cdot a_2^{[1]}$$

**Element wise multiplication*

Derivative of Parameters in 2nd layer

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

backpropagation for binary entropy loss

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$dZ^{[2]} = a^{[2]} - y$$

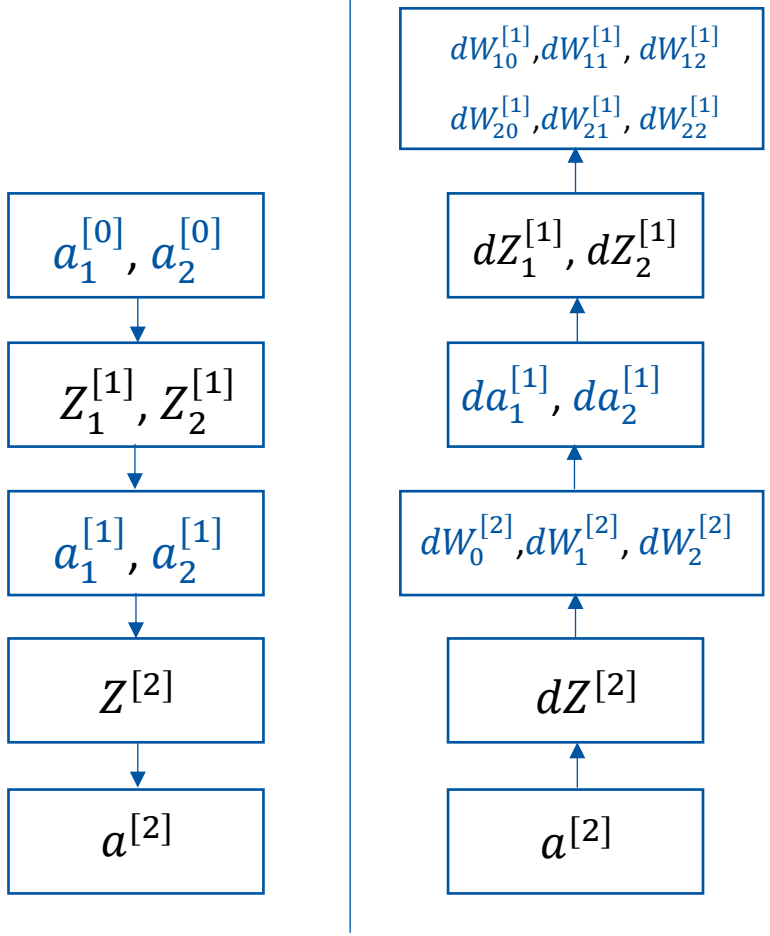
$$dW_0^{[2]} = dZ^{[2]} \cdot 1$$

$$dW_1^{[2]} = dZ^{[2]} \cdot a_1^{[1]}$$

$$dW_2^{[2]} = dZ^{[2]} \cdot a_2^{[1]}$$

Forward propagation

Backward propagation



1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

Cache forward pass variables

To compute the backward pass,

it is very helpful to have some of the variables that were used in the backward pass or forward pass.

In practice you want to structure your code so that you cache these variables,

and so that they are available during backpropagation.

Autograd: PyTorch automatic gradient computation

- For a specific model architecture, we computed the gradient using calculus.
- This approach does not scale to more complex models with millions of weights and perhaps nonlinear complex functions.
- For scalability, we use an automatic differentiation software library like PyTorch Autograd.
- Users of the library need not worry about how to compute the gradients—they just construct the model function.
- Once the function is specified, PyTorch figures out how to compute its gradient through the Autograd technology.

Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Differential equation (Derivative)

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

indicate the rate of change of a function with respect to that variable surrounding an infinitesimally small region near a particular point

$$f(x+h) = f(x) + h\frac{df(x)}{dx}$$

when h is very small,
then the function is well approximated by a straight line,
and the derivative is its slope

Group project schedule

tentative and subject to change

- Feb 04: Group formation (on canvas)
- Feb 18: Project proposal (within 5 mins)
- March 18: Project progress checkup (within 10 mins) – 5 pts
- April 29: Submit presentation slides/ Project presentation (within 20 mins) – 10 pts
- May 6: Submit source codes and report (within 6 pages)– 5 pts